🚀 Hackathon Challenge: Build a Transformer for Next-Word Prediction

Technology Stack: PyTorch, Python 3.8+

📌 Problem Statement

Build a **Transformer-based language model** that predicts the next word in a sequence. Your model must be trained on a provided small dataset and meet specific performance benchmarks through defined checkpoints.

Goal: Implement a working transformer architecture, train it on Shakespeare's text (sample dataset), achieve measurable improvements, and present results.

📊 Dataset Specification

Dataset Details

- **Source:** Shakespeare Corpus (small, standard dataset)
- **Size:** ~5MB, ~40,000 lines of dialogue
- **Vocabulary Size:** ~5,000 unique words (controlled)
- **Train/Val/Test Split:** 70% / 15% / 15%
- **Sequence Length:** 32 tokens
- **Batch Size:** 32

Why This Dataset?

✅ Small enough to train in hours

✅ Standard and reproducible

✅ Real language with clear patterns

🎯 Checkpoints (Must Complete in Order)

✅ Checkpoint 1: Data Preparation

Deliverable: `data_pipeline.py` + `data/` folder

Requirements:

1. Download Shakespeare corpus (provided link)
2. Implement tokenizer (simple word-based OR byte-pair encoding)
3. Create train/val/test splits
4. Build PyTorch Dataset and DataLoader classes
5. Generate sample batch to verify shapes

Success Criteria:

```
# Must produce this output
batch_input.shape == (32, 32)      # (batch_size, seq_lengt
h)
batch_target.shape == (32, 32)     # (batch_size, seq_lengt
h)
vocab_size >= 4000                 # Tokenizer vocab
print("✓ Data pipeline working")
```

Files to Submit:

- `data_pipeline.py` (150-200 lines)
- `tokenizer.pkl` (saved tokenizer)
- `data_stats.json` (vocab size, token counts, etc.)

✅ Checkpoint 2: Transformer Architecture

Deliverable: `transformer_model.py`

Requirements: Implement ONLY these components (no `nn.Transformer`):

1. **ScaledDotProductAttention** (30-40 lines)
   - Q, K, V projections
   - Scaled dot-product: softmax(QK^T/√d_k)V
   - Attention mask support (causal masking)
2. **MultiHeadAttention** (40-50 lines)

- Split into multiple heads
- Combine with output projection
- Head dimension = d_model / num_heads

3. **FeedForwardNetwork** (15-20 lines)
   - Linear → ReLU → Linear
   - Hidden dimension = 4 × d_model
4. **TransformerBlock** (30-40 lines)
   - Attention + Residual + LayerNorm
   - FFN + Residual + LayerNorm
   - Pre-LN architecture
5. **Transformer** (50-60 lines)
   - Token Embedding
   - Positional Encoding (sin/cos)
   - Stack of N transformer blocks
   - Output projection to vocab size

Mandatory Model Config:

```
CONFIG = {
    'vocab_size': 5000,
    'd_model': 128,          # Embedding dimension
    'num_heads': 4,          # MUST divide d_model
    'num_layers': 2,         # Start small for quick traini
ng
    'd_ff': 512,             # 4 × d_model
    'max_seq_length': 32,
    'dropout': 0.1
}
```

Success Criteria:

```
model = Transformer(CONFIG)
batch = torch.randint(0, 5000, (32, 32))
```

```
output = model(batch)
assert output.shape == (32, 32, 5000)  # (batch, seq, voca
b)
print(f"✓ Model parameters: {count_parameters(model):,}")
# Should be ~1-2 million parameters
```

Files to Submit:

- `transformer_model.py` (200-250 lines, fully commented)
- `model_summary.txt` (parameter count, architecture diagram in text)

✅ Checkpoint 3: Training Pipeline

Deliverable: `train.py` + `checkpoints/` folder

Requirements:

1. Implement training loop with:
    - Adam optimizer
    - CrossEntropyLoss (with label smoothing)
    - Learning rate scheduling (warmup + cosine decay)
    - Gradient clipping
2. Validation every N batches
3. Save best checkpoint (based on val loss)
4. Early stopping if no improvement for 3 epochs
5. Log metrics: loss, perplexity, accuracy

Training Hyperparameters:

```
TRAINING_CONFIG = {
    'num_epochs': 10,
    'learning_rate': 0.001,
    'warmup_steps': 500,
    'batch_size': 32,
    'gradient_clip': 1.0,
    'weight_decay': 0.0001,
    'log_interval': 50,      # Log every N batches
```

```
    'val_interval': 200,     # Validate every N batches
}
```

Success Criteria:

```
Epoch 1/10
Batch [50/100]   Loss: 8.234  | Perplexity: 3784.2  | Acc:
0.015
Batch [100/100]  Loss: 7.102  | Perplexity: 1234.5  | Acc:
0.022
Validation Loss: 6.845 | Perplexity: 932.1 | Acc: 0.031
✓ Checkpoint saved: best_model.pt

[After 5+ epochs of training]
Final Validation Loss < 4.5
Final Validation Perplexity < 100
✓ Training complete
```

Files to Submit:

- `train.py` (250-300 lines)
- `checkpoints/best_model.pt` (trained weights)
- `training_log.txt` (complete training output)
- `loss_curves.png` (training vs validation loss plot)

✅ Checkpoint 4: Inference & Next-Word Prediction

Deliverable: `inference.py` + sample predictions

Requirements:

1. Load trained model
2. Implement next-word prediction:
    - Input: sequence of words
    - Output: top-5 predicted next words with probabilities
3. Generate predictions for 10 test examples
4. Show confidence scores

Success Criteria:

```
predictor = NextWordPredictor(model, tokenizer)

# Test input: "To be or not to"
predictions = predictor.predict_next_word("To be or not t
o", k=5)
# Output:
# 1. "be" - 0.342
# 2. "that" - 0.156
# 3. "not" - 0.089
# 4. "the" - 0.067
# 5. "a" - 0.045

print("✓ Predictions generated")
```

Files to Submit:

- `inference.py` (100-150 lines)
- `predictions_sample.txt` (10 examples with top-5 predictions)

✅ Checkpoint 5: Evaluation & Metrics

Deliverable: `evaluate.py` + `results.json`

Requirements: Compute on TEST set:

1. **Loss & Perplexity**

   ```
   test_loss = 3.8
   test_perplexity = 44.7
   ```

2. **Accuracy Metrics**

   ```
   top1_accuracy = 0.18    # % of correct next-word predict
   ions
   top5_accuracy = 0.35    # % where correct word in top-5
   ```

3. **Compare with Baseline**

   - Baseline: Random next-word selection

- Baseline accuracy: ~0.0002 (1/5000)
- Your model should be >>1%

Success Criteria:

```
{
  "test_loss": 3.8,
  "test_perplexity": 44.7,
  "top1_accuracy": 0.18,
  "top5_accuracy": 0.35,
  "baseline_accuracy": 0.0002,
  "improvement_vs_baseline": "900x better"
}
```

Files to Submit:

- `evaluate.py` (100-150 lines)
- `results.json` (final metrics)
- `evaluation_report.txt` (analysis)

✅ Checkpoint 6: Quick Ablation

Deliverable: `ablation_results.txt`

Quick ablations to run (fast, small models):

1. **Num Layers Impact**
   - Train with 1, 2, 3 layers
   - Report final val loss for each
2. **Num Heads Impact**
   - Train with 1, 2, 4 heads
   - Report final val loss for each
3. **Dropout Impact**
   - Train with dropout 0.0, 0.1, 0.3
   - Report final val loss for each

Format:

```
=== ABLATION RESULTS ===

NUM LAYERS:
- 1 layer:   val_loss = 5.2
- 2 layers:  val_loss = 4.1
- 3 layers:  val_loss = 3.9

NUM HEADS:
- 1 head:    val_loss = 4.5
- 2 heads:   val_loss = 4.0
- 4 heads:   val_loss = 3.8

DROPOUT:
- 0.0:       val_loss = 3.6
- 0.1:       val_loss = 3.8
- 0.3:       val_loss = 4.2

FINDINGS:
- More layers improve performance (diminishing returns afte
r 2)
- 4 heads > 2 heads > 1 head
- Optimal dropout = 0.0 on this small dataset
```

📦 Expected Deliverables Summary

Final Submission Folder Structure:

```
transformer-hackathon/
├── README.md                           # Quick start guide
├── requirements.txt                    # Python dependencie
s
├──
├── CHECKPOINT_1_Data/
│   ├── data_pipeline.py              # Data loading code
│   ├── tokenizer.pkl                 # Trained tokenizer
│   ├── data_stats.json               # Vocabulary info
│   └── sample_batch.txt              # Sample data verific
```

```
ation
│
├── CHECKPOINT_2_Model/
│   ├── transformer_model.py          # Architecture (NO HI
GH-LEVEL MODULES)
│   └── model_summary.txt             # Parameter count
│
├── CHECKPOINT_3_Training/
│   ├── train.py                      # Training loop
│   ├── checkpoints/
│   │   └── best_model.pt             # Trained weights
│   ├── training_log.txt              # Full training outpu
t
│   └── loss_curves.png               # Plot of loss over t
ime
│
├── CHECKPOINT_4_Inference/
│   ├── inference.py                  # Next-word predictio
n
│   └── predictions_sample.txt        # 10 example predicti
ons
│
├── CHECKPOINT_5_Evaluation/
│   ├── evaluate.py                   # Evaluation script
│   ├── results.json                  # Final metrics
│   └── evaluation_report.txt         # Analysis
│
├── CHECKPOINT_6_Ablation/  (Optional, Bonus)
│   └── ablation_results.txt          # Ablation findings
│
└── PRESENTATION/
    ├── slides.pdf                    # 5-slide presentatio
n
    ├── demo_notebook.ipynb           # Interactive demo
    └── architecture_diagram.png      # Model visualization
```

🎁 Bonus Points

+20% Bonus: Ablation Studies

Complete Checkpoint 6 with systematic ablations

+15% Bonus: Attention Visualization

- Visualize attention weights for sample inputs
- Show which words attend to which positions
- Create heatmap visualization

+15% Bonus: Novel Architecture Improvement

- Try one modification:
  - Rotary positional embeddings (RoPE)
  - Multi-Query Attention (MQA)
  - Different activation (GELU instead of ReLU)
- Compare performance vs baseline

+10% Bonus: Production Code Quality

- Type hints throughout
- Comprehensive docstrings
- Error handling
- Unit tests (pytest)

+10% Bonus: Demo & Presentation

- Interactive Jupyter notebook
- 5-slide presentation with results
- Live demo during presentation

📋 Evaluation Rubric

Component

Points

Criteria

Checkpoint 1: Data

10

Correct train/val/test, tokenizer works, shapes correct

Checkpoint 2: Model

25

Attention correct, no high-level modules, architecture sound

Checkpoint 3: Training

25

Proper training loop, convergence, checkpointing, logging

Checkpoint 4: Inference

15

Top-5 predictions, probabilities correct

Checkpoint 5: Evaluation

15

Metrics computed correctly, baseline comparison

Code Quality & Documentation

10

Comments, structure, reproducibility

Bonus (Ablation/Viz/etc)

+20

Systematic analysis, visualizations

Total: 100 points + 20 bonus

Minimum to Pass: 60 points (6/10 checkpoints)

📊 Expected Performance Benchmarks

After training for 10 epochs on the provided data:

Metric

Expected Range

Your Target

Final Train Loss

2.5 - 3.5

< 3.0

Final Val Loss

3.5 - 4.5

< 4.0

Val Perplexity

30 - 90

< 55

Top-1 Accuracy

12% - 22%

> 15%

Top-5 Accuracy

25% - 40%

> 30%

Training Time

1-3 hours

Complete in 24hrs

🛠️ Tech Stack Requirements

```
torch>=1.9.0
numpy>=1.19.0
pandas>=1.1.0
matplotlib>=3.3.0
tqdm>=4.50.0
scikit-learn>=0.23.0
```

Optional (for visualizations):

```
seaborn>=0.11.0
```

```
jupyter>=1.0.0
```

📚 Dataset Download

Option 1: Direct Download (Provided)

```
# Download small Shakespeare corpus (5MB)
wget <https://raw.githubusercontent.com/brunoklein99/deep-learning-notes/master/shakespeare.txt>
# Or use provided: data/shakespeare.txt
```

Option 2: Use Included Dataset

```
# Included in submission package
from data.shakespeare import get_shakespeare_data
train, val, test = get_shakespeare_data()
```

Dataset Structure:

```
ROMEO: But, soft! what light through yonder window breaks?
It is the east, and Juliet is the sun.
Arise, fair sun, and kill the envious moon,
...
```

🚀 Submission Checklist

- ☐ All 6 checkpoints complete (at least 5 required)
- ☐ Code is runnable and reproducible
- ☐ All required files present in folder
- ☐ README with setup instructions
- ☐ requirements.txt with all dependencies
- ☐ Training log showing convergence
- ☐ Final results JSON with metrics

- [ ] Presentation ready (5 slides or demo)
- [ ] Code is commented and readable
- [ ] No external transformer libraries used (except for baselines)

❓ FAQ

Q: Can I use nn.Transformer?

A: No. You must implement attention, FFN, and blocks from scratch. You can use nn.Linear, nn.Embedding, nn.LayerNorm, etc.

Q: What if my model doesn't converge?

A: Common fixes:

- Check learning rate (try 0.0005-0.001)
- Verify attention mask (causal masking)
- Reduce model size first for quick iteration
- Check data pipeline (correct shapes?)

Q: How long does training take?

A: ~30 min - 2 hours on CPU, 5-10 min on GPU

Q: Can I use a pre-trained tokenizer?

A: Yes, but document it. Or implement simple word-based tokenization (~50 lines).

Q: What if I don't have a GPU?

A: This dataset trains fine on CPU. Use smaller batches if OOM.

Q: Can I use more data?

A: Not recommended - stick to provided Shakespeare dataset for fair comparison.

🏆 Selection Criteria

1. **All checkpoints working** ✓
2. **Model achieves >15% top-1 accuracy** ✓
3. **Clean, well-documented code** ✓

4. **Presentation demonstrates understanding ✓**
5. **Bonus points for ablations/visualizations ✓**

Good luck! Build fast, break things, iterate quickly. 🚀